Dynamic Analysis and an Eigen Initializer for Recurrent Neural Networks

1st Ran Dou
Department of Electrical and Computer Engineering
University of Florida
Gainesville, FL, United States
dour@ufl.edu

2nd Jose Principe
Department of Electrical and Computer Engineering
University of Florida
Gainesville, FL, United States
principe@cnel.ufl.edu

2023 International Joint Conference on Neural Networks (IJCNN) | 978-1-6654-8867-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/IJCNN54540.2023.10191986

***Abstract*—In recurrent neural networks, learning long-term dependency is the main difficulty due to the vanishing and exploding gradient problem. Many researchers are dedicated to solving this issue and they proposed many algorithms. Although these algorithms have achieved great success, understanding how the information decays remains an open problem. In this paper, we study the dynamics of the hidden state in recurrent neural networks. We propose a new perspective to analyze the hidden state space based on an eigen decomposition of the weight matrix. We start the analysis by linear state space model and explain the function of preserving information in activation functions. We provide an explanation for long-term dependency based on the eigen analysis. We also point out the different behavior of eigenvalues for regression tasks and classification tasks. From the observations on well-trained recurrent neural networks, we proposed a new initialization method for recurrent neural networks, which improves consistently the performance. It can be applied to vanilla-RNN, LSTM, and GRU. We test is on many datasets, such as Tomita Grammars, pixel-by-pixel MNIST dataset, and machine translation dataset (Multi30k). It outperforms Xavier initializer and kaiming initializer as well as other RNN-only initializers like IRNN and sp-RNN in several tasks.**

***Index Terms*—recurrent neural networks, initializer, eigendecomposition, state space model**

I. Introduction

In a recurrent system, the hidden state is considered representative of the unknown state of the system that created the data. In general, a state space model approximates the unknown state h_t using the previous hidden state h_{t-1} and current sample x_t. And the prediction y_t is observed by the hidden state.

$$h_t = H(x_t, h_{t-1}) \tag{1}$$

$$y_t = G(h_t) \tag{2}$$

where H is the state transition function and G is the observation function. Extensive research has been studied on the design of state transition functions, such as the Kalman Filter family [27], echo state networks [13] and recurrent neural networks. The hidden states represent the memory of past information, however, information stored in hidden states is fading away through time. Models have difficulty learning the long-term dependencies.

Many mechanisms have been proposed to improve long-term memory. One is to use the gating mechanism [4], [10], [12] and these gated networks have been widely used in time series, reinforcement learning, and natural language processing. Generally, the output of a gated unit is limited from 0 to 1 by a sigmoid activation function. The forgetting gate is used to reset the hidden state memory and the writing gate decides what information should be stored in the hidden states. In LSTM [12], the forgetting and writing gates operate a cell state vector as the memory. And the GRU operates directly on the hidden state [4]. In order to learn the long-term dependency, a bias of 1 can be added to the forgetting gate for initialization [14].

Another mechanism for improving the long-term dependency is using external memory, such as the Neural Turing Machine (NTM [7]), Differentiable Neural Computer [8], End-to-end Network [23], Neural Stack and Neural Queue [9], RNN-EM [21], and Dynamic Memory Networks [15]. In these architectures, a recurrent network is used as the controller to interact with the external memory. At each time, the controller reads and writes from the external memory based on a rule such as the attention mechanism. Unlike overwriting new information into hidden states, the external memory stores all past information and provides a better long-term dependency.

Besides the two mechanisms mentioned above, there are other studies trying to improve the performance of RNNs. And most of them are focusing on manipulating the hidden state, such as AntisymmetricRNN [3], Noisy RNN [17], PF-RNN [19], SRNN [22], TP-RNN [18], uRNN [2] and SR-RNN [26]. These methods show that well-designed hidden states are capable of handling longer memory. Other methods are focusing more on the learning algorithms for RNNs. For example, [1] shows that with a sufficiently large architecture, the stochastic gradient descent can provide a linear convergence rate. [28] proposed a stochastic bilevel optimization for RNN to prevent vanishing and exploding gradients. And [20] proposed to use the stochastic Riemannian coordinate descent for RNN training.

There is also research studying the effect of different

This work was partially supported by ARO grant W911NF-21-1-0254.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.

initialization methods for RNNs. In gradient descent methods, a proper initialization for weights is of vital importance. If the weights are initialized with small values, the gradients will vanish and if the weights are initialized with large values, the gradients will explode. And the rules of thumb for weight initialization are to set the mean of activations close to zero and the variance of activations consistent with each other. Followed by this idea, Xavier initializer [6] and kaiming initializer [11] are proposed and frequently used in nowadays research. Unlike feedforward neural networks, recurrent neural networks share weights recurrently, which provides a different dynamic for the hidden states. IRNN [16] shows that RNNs can be simply initialized by identity matrices and [24] proposed the np-RNN afterward. However, both methods are designed for vanilla RNNs and can not be applied in general applications such as LSTM and GRU.

In this paper, we start by analyzing a linear state space model in terms of eigendecomposition and extrapolate it to nonlinear cases. We conclude that in RNNs, the long-term dependency is improved by increasing eigenvalues. Based on our observation, we propose a new initializer for recurrent layers. Unlike IRNN and np-RNN, which are only designed for vanilla-RNN, our initializer can be widely used for different types of RNNs, like LSTM and GRU.

II. Eigen Decomposition for State Transition Function

Given a linear state space model, the state transition function and observation function can be written as,

$$h_t = W_h h_{t-1} + W_x x_t \tag{3}$$

$$y_t = W_y h_t \tag{4}$$

where h_t is the state vector with the size of n. For simplicity, denote,

$$x'_t = W_x x_t \tag{5}$$

For the state transition matrix W_h, let $\Lambda = [\lambda_1, \lambda_2, ..., \lambda_n]$ and $U = [u_1, u_2, ..., u_n]$ be the eigenvalues and eigenvectors,

$$W_h = U\Lambda\bar{U}^T \tag{6}$$

Note here $[u_1, u_2, ..., u_n]$ are the orthonormal basis, and Λ and U may have complex values. When the eigenvalues are real values, after each update, the hidden state is enlarged along the direction of eigenvectors by the corresponding eigenvalues. And when the eigenvalues are complex numbers, they show up in conjugate pairs and the hidden state also rotates in the plane formed by the corresponding conjugate eigenvectors.

Consider only the hidden state with zero inputs, we can decompose h_{t-1} by the eigenvectors,

$$h_{t-1} = \sum_{i=1}^{n} \alpha_{t-1}^i u_i \tag{7}$$

where $[\alpha_{t-1}^1, \alpha_{t-1}^2, ..., \alpha_{t-1}^n]$ are the coefficients for the corresponding eigenvectors. And,

$$h_t = \sum_{i=1}^{n} \alpha_t^i u_i = \sum_{i=1}^{n} \alpha_{t-1}^i \lambda_i u_i \tag{8}$$

After t' steps,

$$h_{t+t'} = \sum_{i=1}^{n} \alpha_{t-1}^i \lambda_i^{t'} u_i \tag{9}$$

It is obvious that the system is stable only when $|\lambda_i| \leq 1$. And as time goes on, the information of hidden states is decaying at different speeds along different eigenvectors.

Now take the input x'_t into consideration. Same as the hidden state, we can decompose the x'_t by the eigenvectors u_i and its coefficients a_t^i.

$$x'_t = \sum_{i=1}^{n} a_t^i u_i \tag{10}$$

Set the initial hidden state to zero vector, then we have,

$$h_1 = x'_1 = \sum_{i=1}^{n} a_1^i u_i \tag{11}$$

Compute h_t recursively,

$$h_t = \sum_{i=1}^{n} \sum_{j=1}^{t} a_j^i \lambda_i^{j-1} u_i \tag{12}$$

Recall (8), then we have,

$$\alpha_t^i = \sum_{j=1}^{t} a_j^i \lambda_i^{j-1} \tag{13}$$

$$\alpha_t^i = \lambda_i \alpha_{t-1}^i + a_t^i \tag{14}$$

Therefore, a linear state space model equals a set of first-order IIR filters in complex space along the direction of eigenvectors.

In nonlinear recurrent systems, for the weight matrix of the hidden state, we can still use the same analysis of eigendecomposition as above. However, equation (12) breaks due to the nonlinear activation function, causing it difficult to analyze nonlinear cases.

III. Conjectures for Nonlinear Case

The activation functions are used to introduce non-linearity in recurrent neural networks. Here, we can rethink it from the view of eigenvalues and eigenvectors of the weight matrix in equation (9). However, the issue is that the information decay of a hidden state is determined by the eigenvalues. A system with small eigenvalues loses information very fast. And in a stable linear system, the eigenvalue norms are strictly limited by 1, which provides no long-term dependency. However, using nonlinear activation functions restricts the outputs either from 0 to 1 ($sigmoid$) or from -1 to 1 ($tanh$). It prevents the hidden states from exploding when the eigenvalues are greater than 1. As for the $ReLU$ activation, there are cases where eigenvalues are negative numbers, meaning the hidden states flip in the opposite direction. And $ReLU$ prevents the exploding by directly cutting the negative parts. Hereby, we make the following conjecture:

Conjecture 1: A nonlinear recurrent system improves the long-term dependency by increasing the eigenvalues of the

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.

weighting matrix (> 1). The larger the eigenvalues are, the better long-term dependency is provided.

To test this conjecture, we perform two experiments on linear-RNN, tanh-RNN, and relu-RNN and compare the eigenvalue norms. We first train the three models on a regression task using the Mackey glass dataset. We use the MSE as the loss function and Adam optimizer with a learning rate of 0.01. We set the size of the hidden state to 8. Since the new hidden state is given by both the previous hidden state and input sample, the eigenvalue norms should be much smaller than 1. From Table I we can see both tanh-RNN and relu-RNN have larger eigenvalue norms than linear-RNN.

TABLE I
EIGENVALUE NORMS ON MACKEY GLASS

linear-RNN	tanh-RNN	relu-RNN
0.57	0.72	0.66
0.57	0.72	0.52
0.36	0.65	0.52
0.36	0.52	0.50
0.35	0.52	0.50
0.35	0.34	0.25
0.23	0.27	0.25
0.07	0.20	0.04

We also train the three networks on the sequential MNIST classification task, which requires a higher demand for long-term dependency. We use the softmax output and cross-entropy as the loss function. Since models are only trained with fixed length sequences and there is no limitation on the values of the outputs (before the softmax), the restriction for stability in the networks is looser. And the eigenvalue norms can be slightly larger than 1 in linear RNN. Besides, in most classification cases, the models extract features from the input and keep information. Larger eigenvalue norms are expected. We set the size of the hidden state to 150 and use Adam with a learning rate of 0.0001. We only show the largest 10th unique eigenvalue norms (only keep one for conjugate eigenvalues) in Table II.

TABLE II
EIGENVALUES NORMS ON MNIST CLASSIFICATION

linear-RNN	tanh-RNN	relu-RNN
1.02	1.71	1.35
1.01	1.60	1.31
0.99	1.53	1.30
0.98	1.41	1.29
0.98	1.27	0.98
0.96	1.19	0.88
0.95	0.95	0.81
0.94	0.90	0.74
0.91	0.89	0.70
0.83	0.74	0.68

In a stable linear system, the eigenvalue norms cannot be larger than 1, meaning that the information can hardly be preserved in the hidden states. Therefore, the hidden states tend to shrink to zero when propagating forward through time. And the hidden states have a higher density distribution around the origin. It causes discriminating information much more difficult. However, with the nonlinear activation functions, the eigenvalue norms can be greater than 1. Here the space of hidden states is a hypercube limited by the activation functions. These eigenvalues keep pushing the hidden states to the surface of the hypercube. Therefore, the hidden states can explore more space in the hypercube and have a better distribution for discriminant features. In conclusion, we have the following conjecture:

Conjecture 2: For a recurrent system with long-term dependency, the hidden states are capable of exploring the space away from the origin and avoiding collapsing.

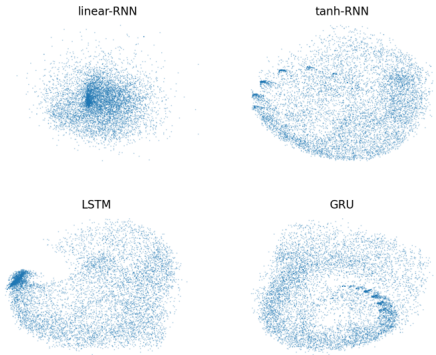


Fig. 1. Distribution of Hidden States

From the tables above, we notice that the classification task has much better long-term dependency than regression. Therefore, we validate this conjecture using the MNIST dataset and compare the hidden states of linear-RNN, tanh-RNN, LSTM and GRU. We first train the four networks until the best performance is achieved. Then, we collect all the hidden states along the trajectories. For visualization, we use the principal component analysis (PCA) and plot the scatters on the first two principal components (Figure 1). We can see that the plots are consistent with our conjecture. Besides, LSTM and GRU show special structures in the distribution which requires future explanation.

IV. EIGEN INITIALIZER

In gradient descent methods, a proper initialization for the weights not only helps with the convergence but also provides better local optimal solutions. To study the relation between eigenvalues and different initialization methods, we calculate the eigenvalue norms for different initializers (Table III). We create a weighting matrix with a size 8×8 and compare the default uniform initializer (in Pytorch) with sp-RNN, Xavier initializer, and kaiming initializer. We generate each initializer 500 times and calculate the ordered mean values. The default initializer of Pytorch is initialized by a uniform distribution with a standard deviation given by,

$$stdv = 1/sqrt(n) \tag{15}$$

where n is the size of input features.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EIGENVALUE NORMS FOR DIFFERENT INITIALIZERS

Default	sp-RNN	Xavier normal	Xavier uniform	kaiming normal	kaiming uniform
0.61±0.09	1.00±0.00	1.06±0.18	1.07±0.16	1.51±0.26	1.51±0.23
0.54±0.07	0.74±0.12	0.95±0.16	0.96±0.13	1.33±0.22	1.35±0.20
0.48±0.07	0.66±0.10	0.83±0.14	0.84±0.12	1.17±0.19	1.19±0.17
0.42±0.07	0.47±0.08	0.73±0.13	0.75±0.12	1.04±0.19	1.06±0.18
0.37±0.07	0.42±0.06	0.63±0.14	0.66±0.12	0.90±0.18	0.92±0.19
0.31±0.08	0.31±0.05	0.52±0.14	0.54±0.14	0.74±0.19	0.77±0.20
0.24±0.09	0.31±0.05	0.39±0.15	0.42±0.15	0.56±0.19	0.59±0.21
0.14±0.09	0.30±0.04	0.24±0.16	0.24±0.15	0.34±0.22	0.35±0.21

From table III, we notice that compared with other initializers, the default initializer provides small eigenvalues. At the beginning of training, if the eigenvalue norms are small, the information of input samples decays very fast. And the model can hardly learn from previous samples. Other initializers provide larger eigenvalues, helping the model to learn from more samples. From the perspective of eigenvalues, we can also explain why other initializers help with the convergence rate.

Therefore, we can help models learn better long-term dependency with a better initialization on eigenvalues. First, we set all the eigenvalue norms to $\lambda \in (0, 1)$. We want the model can learn more from long-term dependency and let the hidden states store more information in the beginning. Therefore, we set λ to 0.95 for all experiments.

$$W_0 = diag(\lambda) \tag{16}$$

where w_1 is a diagonal matrix with the size of $n \times n$, and n is the size of hidden states.

In hidden state space, the transition function cannot only magnify the hidden states but should also rotate the hidden states by a certain angle along several directions. Therefore, we mimic this rotation by decomposing the process into $n-1$ step. And at step i, we randomly sample an angle θ_i from a uniform distribution $[0, 2\pi]$. Then, we perform the rotation in the space of ith and $(i+1)$th dimension.

$$W_i = \begin{bmatrix} 1 & & & & 0 \\ & \ddots & & & \\ & & \cos\theta_i & -\sin\theta_i & \\ & & \sin\theta_i & \cos\theta_i & \\ 0 & & & & 1 \end{bmatrix} \tag{17}$$

Then, the final initializer is,

$$W = \prod_{i=0}^{n-1} W_i \tag{18}$$

V. EXPERIMENTS AND RESULTS

A. Tomita Grammar

The Tomita Grammar [25] is a set of strings with alphabet $\{0, 1\}$ following predefined rules. It contains 7 different languages and has been widely studied for deterministic finite automata (DFA) extraction. Here, we use the Tomita Grammar to test our initialization method. We compare the learning curves on tanh-RNN, LSTM and GRU with uniform initialization (Pytorch), Xavier initializer and kaiming initializer. We also compare IRNN, sp-RNN for tanh-RNN, which are both RNN-only initializers. We show the comparison of learning curves on grammar 4. To show stability, we train each method 20 times and plot the average learning curve. We use Adam as the optimizer and uniformly set the learning rate to 0.001 with no weighting decay.

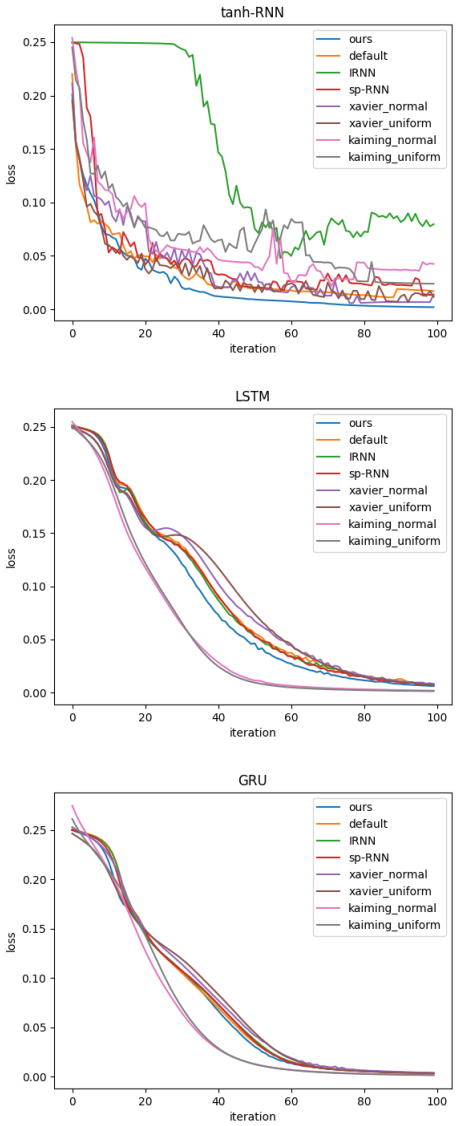


Fig. 2. Average Learning Curves on Tomita Grammar 4

Figure 2 shows that in tanh-RNN, our initialization method provides a smoother learning curve. This is because our initialization method provides a better long-term dependency at the beginning and networks can benefit from this initialization. And in LSTM and GRU, our method shows slight improvement compared with other methods. Note that the Kaiming method takes advantage of the nonsaturating nonlin-

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.

earity (ReLU) and is therefore not comparable to the others.

B. MNIST Classification

We also test our initialization method on MNIST classification. In the experiment, we use the sequential MNIST by scanlines. Each time 28 pixels from the same row are presented as the input and there are 28 inputs in total. This is a simple task and we only care about the convergence of the learning curves. We set the same comparison as in section V-A. We set the hidden states for all networks to 150 and use cross-entropy as the loss function. We use Adam as the optimizer with a learning rate of 0.0001. The result is shown in Figure 3. Our method provides both a good convergence rate and better local optima for all networks. Still, we only have a significant improvement on the convergence rate on tanh-RNN. But in LSTM and GRU, we also benefit from learning the long-term dependency.

C. Multi30k Machine Translation

The Multi30k [5] is a multi-model dataset that has been widely used in machine translation and image description. Each sample contains an image and a pair of descriptions in German and English. In this paper, we only take the sentences and perform a machine translation task from German to English. In the experiment, we choose the encoder-decoder architecture for sequence-to-sequence prediction. Each encoder and decoder contains an embedding layer and a recurrent layer (either tanh-RNN, LSTM or GRU). For simplicity, we only use one layer of each recurrent layer. For data prepossessing, we use spaCy to extract tokens from the sentences, which is an industrial package for natural language processing. Then, we replace all tokens that appear only once with the unknown token [UNK]. We also use the [PAD] token for padding the sequences. We uniformly set the embedding size to 512 and the hidden size to 256. We compare our methods with other initialization methods on tanh-RNN, LSTM and GRU. We use the cross-entropy as the loss function with a label smooth of 0.1. We also use Adam as the optimizer with a learning rate of 0.1. The learning curves are shown in Figure 4. We overtrain the LSTM and GRU cases to display the full convergence of the other methods.

In the tanh-RNN, our method outperforms default uniform, IRNN and sp-RNN. It also provides a better and more stable solution compared with all other initialization methods. In LSTM, our methods provide the best convergence rate as the default uniform initialization and in GRU, our performance is pretty close to all other methods, except Xavier initializers fail to provide a good convergence.

VI. Conclusion

In this paper, we analyze the dynamics of hidden states in recurrent systems. We study the relationship between eigenvalues and long-term dependency in linear recurrent models and testify to the conjectures for nonlinear systems. We also propose a new eigen initializer for all kinds of recurrent neural networks. We test is on multiple datasets. Our results show that

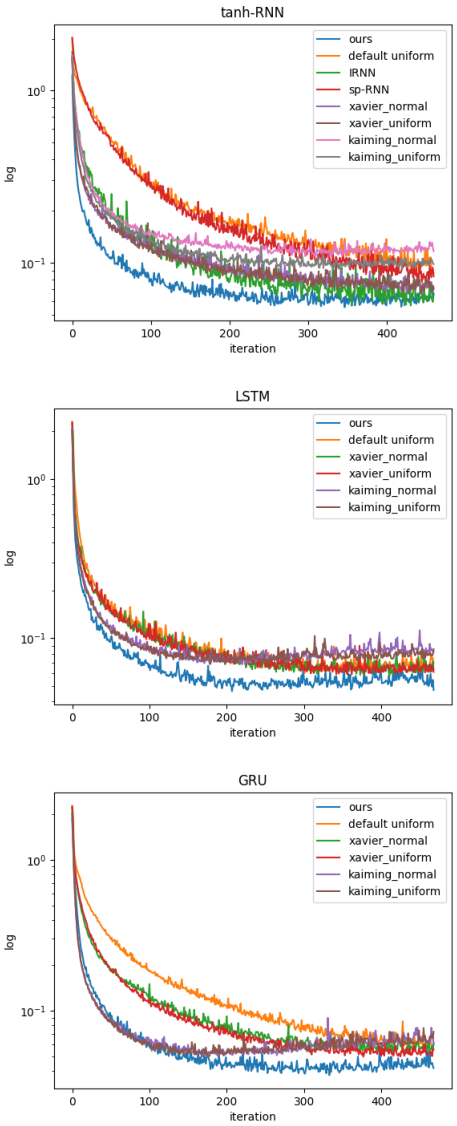


Fig. 3. Learning Curves on scanline MNIST

our initialization methods can not only provide a good convergence rate but can also provide a better local optimal solution. In some experiments, we outperform Xavier initializer and kaiming initializer. We provide a new alternative initializer for training recurrent neural networks and an intuitive explanation of why it works.

References

[1] Z. Allen-Zhu, Y. Li, and Z. Song, "On the convergence rate of training recurrent neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[2] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *International conference on machine learning*. PMLR, 2016, pp. 1120–1128.

[3] B. Chang, M. Chen, E. Haber, and E. H. Chi, "Antisymmetricrnn: A dynamical system view on recurrent neural networks," *arXiv preprint arXiv:1902.09689*, 2019.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.

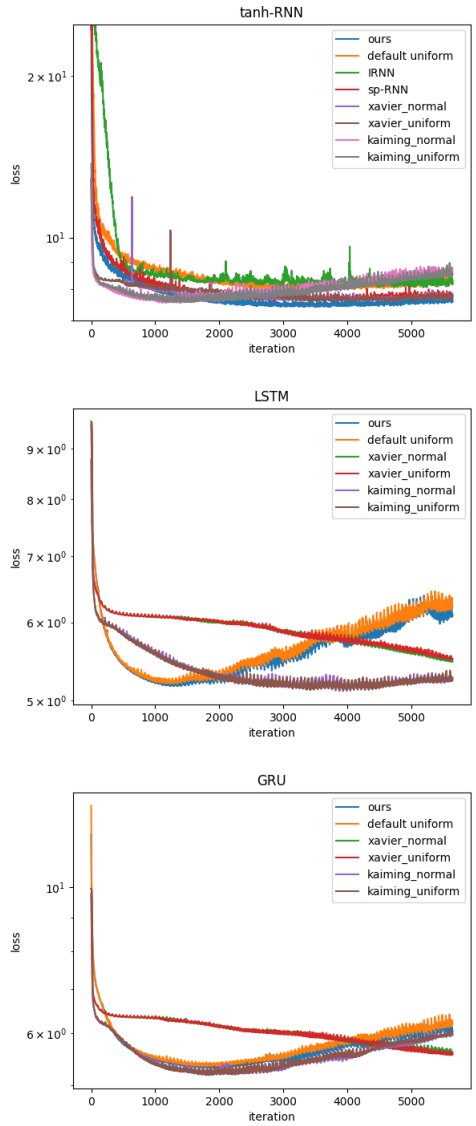


Fig. 4. Learning Curves on Multi30k

[4] K. Cho, B. van Merrienboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, A. Moschitti, B. Pang, and W. Daelemans, Eds. ACL, 2014, pp. 1724–1734. [Online]. Available: https://doi.org/10.3115/v1/d14-1179

[5] D. Elliott, S. Frank, K. Sima'an, and L. Specia, "Multi30k: Multilingual english-german image descriptions," *CoRR*, vol. abs/1605.00459, 2016. [Online]. Available: http://arxiv.org/abs/1605.00459

[6] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2010, pp. 249–256.

[7] A. Graves, G. Wayne, and I. Danihelka, "Neural turing machines," *arXiv preprint arXiv:1410.5401*, 2014.

[8] A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwińska, S. G. Colmenarejo, E. Grefenstette, T. Ramalho, J. Agapiou *et al.*, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016.

[9] E. Grefenstette, K. M. Hermann, M. Suleyman, and P. Blunsom, "Learning to transduce with unbounded memory," *Advances in neural information processing systems*, vol. 28, 2015.

[10] A. Gu, C. Gulcehre, T. Paine, M. Hoffman, and R. Pascanu, "Improving the gating mechanism of recurrent neural networks," in *International Conference on Machine Learning*. PMLR, 2020, pp. 3800–3809.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] H. Jaeger, "Adaptive nonlinear system identification with echo state networks," *Advances in neural information processing systems*, vol. 15, 2002.

[14] R. Jozefowicz, W. Zaremba, and I. Sutskever, "An empirical exploration of recurrent network architectures," in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, F. Bach and D. Blei, Eds., vol. 37. Lille, France: PMLR, 07–09 Jul 2015, pp. 2342–2350. [Online]. Available: https://proceedings.mlr.press/v37/jozefowicz15.html

[15] A. Kumar, O. Irsoy, P. Ondruska, M. Iyyer, J. Bradbury, I. Gulrajani, V. Zhong, R. Paulus, and R. Socher, "Ask me anything: Dynamic memory networks for natural language processing," in *International conference on machine learning*. PMLR, 2016, pp. 1378–1387.

[16] Q. V. Le, N. Jaitly, and G. E. Hinton, "A simple way to initialize recurrent networks of rectified linear units," *arXiv preprint arXiv:1504.00941*, 2015.

[17] S. H. Lim, N. B. Erichson, L. Hodgkinson, and M. W. Mahoney, "Noisy recurrent neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 5124–5137, 2021.

[18] Q. Ma, Z. Lin, E. Chen, and G. Cottrell, "Temporal pyramid recurrent neural network," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5061–5068.

[19] X. Ma, P. Karkus, D. Hsu, and W. S. Lee, "Particle filter recurrent neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5101–5108.

[20] E. Massart and V. Abrol, "Coordinate descent on the orthogonal group for recurrent neural network training," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7744–7751.

[21] B. Peng, K. Yao, L. Jing, and K.-F. Wong, "Recurrent neural networks with external memory for spoken language understanding," in *Natural Language Processing and Chinese Computing*, J. Li, H. Ji, D. Zhao, and Y. Feng, Eds. Cham: Springer International Publishing, 2015, pp. 25–35.

[22] M. Rotman and L. Wolf, "Shuffling recurrent neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 11, 2021, pp. 9428–9435.

[23] S. Sukhbaatar, J. Weston, R. Fergus *et al.*, "End-to-end memory networks," *Advances in neural information processing systems*, vol. 28, 2015.

[24] S. S. Talathi and A. Vartak, "Improving performance of recurrent neural network with relu nonlinearity," *arXiv preprint arXiv:1511.03771*, 2015.

[25] M. Tomita, "Dynamic construction of finite automata from examples using hill-climbing," in *Proceedings of the Fourth Annual Conference of the Cognitive Science Society*, Ann Arbor, Michigan, 1982, pp. 105–108.

[26] C. Wang and M. Niepert, "State-regularized recurrent neural networks," in *International Conference on Machine Learning*. PMLR, 2019, pp. 6596–6606.

[27] D. WHITNEY, "The kalman filter family tree- a survey of state-of-the-art analysis methods based on the kalman filter," *NAECON 1984*, pp. 426–432, 1984.

[28] Z. Zhang, Y. Yue, G. Wu, Y. Li, and H. Zhang, "Sbo-rnn: Reformulating recurrent neural networks via stochastic bilevel optimization," *Advances in Neural Information Processing Systems*, vol. 34, pp. 25 839–25 851, 2021.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:10 UTC from IEEE Xplore. Restrictions apply.